Towards Perfect Code Generation: Iterative API-based Feedback Loops

Harman Preet Singh
852863316

Abstract

The abstract paragraph should be indented ½ inch (3 picas) on both the left- and right-hand margins. Use 10 point type, with a vertical spacing (leading) of 11 points. The word **Abstract** must be centered, bold, and in point size 12. Two line spaces precede the abstract. The abstract must be limited to one paragraph.

Keywords: keyword 1 · keyword 2 · keyword 3

1 Introduction (1 – 1.5 pages)

Introduce the problem: LLMs generating syntactically valid but semantically poor HTML (the "divitis" problem). Explain why this matters — maintainability, accessibility, screen readers. End with a brief statement of your research questions and a roadmap of the report. Keep in mind: This is where you frame the motivation clearly. Don't go too deep into related work yet, but do establish that this is a real and known problem.

2 Related Work (2 – 3 pages)

This is the section most notably missing from the proposal. Cover:

- Existing work on LLM code quality and hallucinations
- Prior research on iterative feedback / self-correction loops for LLMs (e.g. the paper your supervisor linked: https://arxiv.org/html/2603.23613v1 — which uses multiple feedback models)
- Prior work on automated code validation as a guardrail
- Any existing evaluations of HTML semantic quality

Keep in mind: Your supervisor explicitly flagged the absence of related work. This section also gives you the opportunity to position your contribution clearly — what does your approach do differently or better?

3 Research Questions & Hypotheses (0.5 pages)

Formally state RQ1 and RQ2. You can also add brief hypotheses based on your initial testing (e.g. reasoning models converge faster).

4 Methodology (4 – 5 pages)

4.1 System Architecture

Describe the three components (LLM, validator, reprompting pipeline) and how they interact. Include your flowchart (Figure A1). Explain design choices, e.g. why Docker-hosted W3C rather than the public API.

4.2 Prompt Dataset

Describe how you constructed your prompt set. How many prompts? How did you decide which ones to include? Did you deliberately design some to provoke semantic tag errors (e.g. tasks that structurally require `<nav>`, `<article>`, etc.)? This directly addresses the supervisor's feedback about the random prompt selection being unclear.

Change: Remove the mention of random sampling — the dataset is now a fixed, ordered set of 51 prompts across three difficulty tiers (simple / medium / difficult). Describe how prompts were designed across tiers, and that the fixed ordering ensures apples-to-apples model comparisons.

4.3 Models & Configurations

List all models tested. Describe the parameter ranges (size, quantisation). Explain how you vary the maximum iteration count and how you determine when you've found the "optimal" cut-off — frame this as a quality-vs-cost tradeoff that you measure empirically.

Change: Expand this significantly. You now have a concrete model table to discuss (gemma4:e2b, qwen3:8b, qwen2.5-coder:14b, qwen3.5:9b) with an interesting emerging finding: reasoning capability matters more than parameter count. qwen2.5-coder:14b has more parameters than the others combined, yet it fails to fix issues because it lacks reasoning. This is now a key variable alongside model size. Add `temperature` and deterministically derived `seed` as explicit configuration parameters worth mentioning for reproducibility.

4.4 Validation Strategy

This is the most critical methodological section. Address the supervisor's concern directly: can the W3C validator actually detect semantic misuse of ? If it cannot (likely the case — W3C validates syntax, not semantics), you need to be transparent about this limitation and introduce a complementary evaluation. Options to discuss:

- A human evaluation rubric (e.g. annotators scoring semantic correctness)
- A secondary automated check (e.g. counting occurrences of semantic tags vs. before and after)
- Possibly a separate LLM-as-judge evaluation

Clarify what "comparable output quality" means for RQ2 — define your quality metric explicitly.

Change: Add a subsection on the blind ablation control. This is a significant addition — for each run, the pipeline now runs both a `feedback` condition (model sees validator errors) and a `blind` condition (model is only told to "review and fix"). This lets you isolate how much improvement comes from the validator feedback itself versus simply giving the model a second attempt. This is important to call out as a deliberate design choice.

Also add: `check_validator_determinism.py` is used to verify the W3C validator itself is stable before trusting any before/after deltas, worth mentioning as a validity check.

4.5 Cost Metric

Define how you operationalise "cost" — API pricing per token, latency, or total tokens consumed per run. This addresses the supervisor's question about RQ2's cost dimension.

Change: Clarify that cost is proxied via generation latency and token counts (not API pricing), since all models run locally through Ollama. The results schema captures `gen_time_s`, `prompt_eval_count`, and `eval_count` per iteration. Note explicitly that no cloud/API-based model baseline is included in this pipeline (as stated in the repo's Scope section) — this is a scope limitation to acknowledge.

5 Experimental Setup (1 – 1.5 pages)

Describe the practical setup: hardware, software versions, Docker configuration, Ollama setup, which specific model versions/quantisations you used, and how you controlled for variability between runs. Include any pilot/exploratory results that informed your final setup (you already have some initial results with Gemma 4, Qwen 3, DeepSeek R1).

Change: Add mention of repeated trials per (model, prompt) combination, which enables variance estimation and confidence intervals rather than relying on single-run results. Also mention the `was_cleaned` flag, the pipeline strips markdown fences/preamble before validation, so formatting slip-ups are never counted as HTML errors (important for validity of the before/after comparison).

6 Results (3 – 4 pages)

6.1 RQ1 — Guardrail Effectiveness

Report error/warning counts before and after the guardrail, per model and per prompt. Show whether the guardrail improves semantic correctness — and be careful to distinguish between W3C validation errors and semantic correctness (flag this distinction explicitly here).

Change: Results now need to be broken down by condition (`feedback` vs `blind`) as well as by model and difficulty tier, not just before/after. The statistical analysis uses paired Wilcoxon tests and 95% confidence intervals, which should be stated. Also add error category breakdown as a dimension (missing-doctype, stray-tag, disallowed-attribute, etc.).

6.2 RQ2 — Cost-Efficiency

Compare smaller vs. larger models on quality metrics at various iteration counts. Show the quality-vs-cost tradeoff curve. Establish a baseline for large API-based models (or compare against a cited benchmark) as your supervisor suggested.

Change: Reframe this — the comparison is smaller vs. larger local models only, not local vs. cloud. Add the reasoning vs. non-reasoning dimension as a key finding here, since it turns out to be more predictive of success than parameter count alone. The scatter plots (model size vs. cost/quality, coloured by native thinking mode) directly address this.

6.3 Iteration Count Analysis

When does the loop converge? Is there a diminishing return after N iterations? This is where you answer the "optimal number of iterations" question.

Change: No major structural change, but the analysis now comes from `stats.py`'s iteration-cutoff analysis, which is worth naming explicitly as the source of the convergence findings.

7 Discussion (2 – 3 pages)

8 Conclusion (0.5 – 1 page)

References

Appendix

Full prompt dataset

Extended results tables

Flowchart (if not in main body)

Human evaluation rubric (if used)

Project timeline

- Full prompt dataset
- Extended results tables
- Flowchart (if not in main body)
- Human evaluation rubric (if used)
- Project timeline — your supervisor asked for this; an appendix is a natural place for if it doesn't fit in the main body